Niord Development Guide (Niord v4)

Table of Contents

1. Introduction . . . 1
   1.1. The base product and its countries . . . 1
   1.2. The parts of the repository . . . 1
   1.3. The stack . . . 3
   1.4. The public site and the old niord-proxy . . . 3
   1.5. Integrating with Niord . . . 3
2. Running Niord locally . . . 5
   2.1. Prerequisites . . . 5
   2.2. The one command . . . 5
   2.3. Ports and addresses . . . 6
   2.4. The logins of the local realm . . . 6
   2.5. Other ways to run it . . . 7
3. Backend conventions . . . 8
   3.1. Modules and packages . . . 8
   3.2. Classes and naming conventions . . . 8
   3.3. Localisation . . . 9
   3.4. Converting between entities and value objects . . . 10
   3.5. Services, controllers and injection . . . 11
   3.6. Transactions . . . 11
   3.7. Security . . . 11
   3.8. Errors . . . 12
   3.9. Settings and Spring properties . . . 12
   3.10. Database, JSON and time . . . 12
4. Testing the backend . . . 14
   4.1. Unit tests and integration tests . . . 14
   4.2. Running the tests . . . 14
   4.3. Writing tests . . . 15
   4.4. The tests that guard the rules . . . 15
5. Frontend architecture and conventions . . . 16
   5.1. A map of the workspace . . . 16
   5.2. The layers of the administration application . . . 16
   5.3. Where the conventions are written down . . . 17
   5.4. Everyday commands . . . 18
6. Code, data and configuration . . . 19
   6.1. Three kinds of thing . . . 19
   6.2. What a change to the repository may do to a deployment . . . 19
   6.3. The bootstrap of a new deployment . . . 19
   6.4. Release folders and a system administrator’s import . . . 20

6.5. Defaults, the sample and the repository files . . . 21
7. Building a country . . . 22
7.1. What a country repository holds . . . 22
7.2. Where the rules are . . . 22
8. Releases, images and deployment . . . 24
8.1. What a release consists of . . . 24
8.2. The images . . . 25
8.3. The Helm chart . . . 25
8.4. Deployment . . . 26
9. Contributing . . . 27
9.1. Branches and pull requests . . . 27
9.2. Continuous integration . . . 27
9.3. The gates . . . 28
9.4. Local verification . . . 28
9.5. Licence . . . 28

Chapter 1. Introduction

Niord (Nautical Information Directory) is a system for producing and publishing Navigational Warnings (NW) and Notices to Mariners (NM). Editors write and publish messages in an administration application, mariners read them on a public website, and a backend stores the messages and sends them out by mail and other channels. The Danish Emergency Management Agency is the organisation behind Niord.

This guide is for developers of Niord version 4. It is short on purpose: it explains how the system is put together and which rules the code follows, and for the details it names the documents in the source repository that are kept up to date with the code. Where this guide and such a document disagree, the document in the repository is right.

The source code of Niord 4 lives in one repository, which is not public yet; it is to move to the NiordOrg organisation on GitHub (https://github.com/NiordOrg). This guide therefore names files as paths relative to the root of that repository, such as `be/README.md`.

Earlier versions have guides of their own, kept as an archive: Niord v3 (Quarkus) and Niord v2 (Wildfly). Niord 4 replaces both.

1.1. The base product and its countries

Niord 4 is split into a *base product* and *countries*.

- The **base product** is the whole system, the same for every country and knowing none of them. It is the repository this guide describes. It is released in numbered versions, `4.0.<n>`: a merge to the main branch that changes code and passes its checks becomes a new version, while a merge that changes only documentation publishes nothing of its own (Releases, images and deployment).
- A **country** is a repository of its own, built on one pinned version of the base product. It adds only what makes its deployment its own: its configuration, branding, languages, texts and starting data, and at most a little code through the extension points the base product offers. It never edits the base product's code. On the backend it may not replace a class of the base product at all; on the frontend it may, where no extension point serves, replace a single file with its own copy, a guarded override that the build checks again whenever the base product changes the original (What a country repository holds). The Danish and Greenland deployments are two such countries.

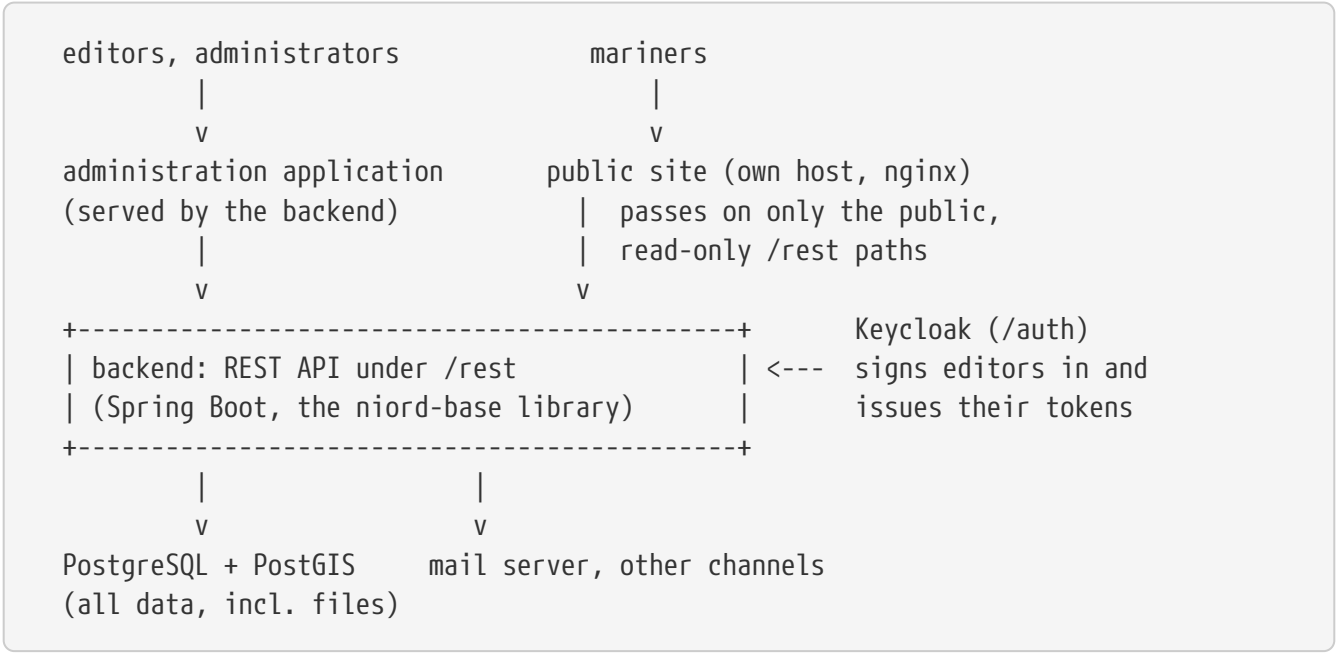
A fix or a new feature made in the base product reaches every country when that country moves to the new version, and a country's own choices stay in its repository through every upgrade. How a country is built is the subject of Building a country.

1.2. The parts of the repository

Path	What
be/	The backend: a Maven reactor with the parent org.niord:niord-parent (be/pom.xml) and three modules. be/niord/ is the base library, org.niord.base:niord-base, which holds all the backend code. be/niord-bootstrap/ packages the data a new deployment starts with (be/niord/bootstrap/) as a zip. be/niord-app/ is the generic English application built on the library, the reference deployable.
be/k8s/	The container image of the backend (be/k8s/docker/Dockerfile), its build scripts and the Helm chart that deploys a complete Niord stack (be/k8s/helm/niord/).
fe/	The frontend: one Angular workspace with two applications. fe/ui-apps/admin/ is the administration application (Angular project niord-app), used by editors, administrators and system administrators after they sign in. fe/ui-apps/info/ is the public site for mariners (Angular project niord-info), anonymous and read-only, shipped in a container of its own.
country-kit/	The country kit: the set-up, the tools and the checks a country repository builds its frontend with, the country guides, and an example country. It is published as a package of its own.
docs/	Plans and records of decisions taken across the repository.
guides/	The sources of this site.

The two halves have their own detailed documents: be/README.md and be/AGENTS.md for the backend, fe/README.md, fe/AGENTS.md and fe/docs/ for the frontend, and country-kit/README.md with country-kit/docs/ for the kit. The AGENTS.md files are written as working rules for anyone changing the code, people and coding assistants alike.

How the parts work together in a running deployment:



1.3. The stack

The backend:

- Java 25 and Spring Boot 4 with Spring MVC. Every REST handler is a `@RestController` method mapped under `/rest/...`.
- PostgreSQL 18 with PostGIS. Liquibase builds and changes the schema at start-up; Hibernate ORM 7 then only validates it. Geometries are handled with Hibernate Spatial and GeoTools.
- Jackson 3 for JSON (the packages `tools.jackson.*`; the annotations are still `com.fasterxml.jackson.annotation.*`).
- Keycloak for sign-in. The backend validates the tokens Keycloak issues and uses Keycloak's admin API to manage users, groups and the client of each domain.
- JBeret for batch jobs (imports and similar long-running work), FreeMarker for templates, reports and mails, a full-text message index (Lucene by default; the `disableLucene` setting switches it to PostgreSQL's full-text search), and an embedded ActiveMQ Artemis broker.

The frontend:

- Angular 22 with standalone components, without zone.js, built on signals and Signal Forms; TypeScript 6 in strict mode.
- Tailwind CSS 4 with the workspace's own design tokens, `@angular/cdk` and `@angular/aria` for behaviour and Lucide icons. No component library is used, and none is to be added.
- OpenLayers 10 for maps, Transloco 8 for translations, TipTap 3 and CodeMirror 6 for rich text and code, `keycloak-js` 26 for sign-in.
- Vitest 4 for unit tests, Playwright for end-to-end tests, ESLint and Prettier for code quality.

1.4. The public site and the old niord-proxy

Earlier versions showed the active messages to mariners through a separate Java web application, niord-proxy. Niord 4 has no such application: the public site, `fe/ui-apps/info/`, replaces it. It is an Angular application served by nginx in a container of its own. The container passes only a fixed list of public, read-only `/rest` paths on to the backend and strips every credential on the way (the list is in `fe/ui-apps/info/AGENTS.md`).

The Helm chart still has a component called `proxy`, whose service is named `niord-proxy` in a release named `niord` (other releases put their own name in front, The Helm chart). It is an nginx reverse proxy in front of the backend and Keycloak (The Helm chart), and it has nothing to do with the old application.

1.5. Integrating with Niord

This guide does not cover integration. A system that reads messages from Niord uses the public REST API under `/rest/public/v1`, which needs no sign-in. It is described in three documents on this site:

- Niord Public API: the endpoints and their parameters.
- Niord Model: the message model the API returns.
- Niord Message Presentation: how a client presents the messages to mariners.

Chapter 2. Running Niord locally

One script at the root of the repository, `start-local.sh`, starts the whole stack on your machine. This chapter says what it starts and where to find it. The repository's `README.md` (sections *Try it* and *Running locally*), `be/README.md` and `fe/README.md` hold the details and every option; read them rather than this chapter when something does not behave as described.

2.1. Prerequisites

- A JDK 25: `JAVA_HOME` pointing at it, or Temurin 25 unpacked into `~/.jdks/temurin-25`.
- Node 24 (the version `fe/.nvmrc` names) with npm 11.
- Docker Desktop, running.
- A bash shell. On Windows, use Git Bash.

Maven need not be installed: the backend comes with the Maven wrapper, `be/mvnw`. Before the first start, or when a start fails for no reason you can see, `be/dev doctor` checks the JDK, Docker, the Docker endpoint the integration tests use and the ports the local services need. It only reads; it changes nothing.

2.2. The one command

From the root of the repository:

```
./start-local.sh --sample          # the sample deployment: both
                                   # applications, sample content
./start-local.sh                   # no content but the bootstrap;
                                   # the administration application
./start-local.sh --info            # ... with the public site instead
./start-local.sh --admin --info    # ... with both
./start-local.sh --sample --reset  # the sample deployment, from an empty database
again
```

The first run builds the backend, which takes a few minutes. The script then:

1. starts three containers from `be/docker-compose.yml`: PostgreSQL with PostGIS (`niord-db`), Keycloak with the local realm (`niord-keycloak`) and MailHog, which catches every mail the backend sends (`niord-smtp`). The containers are shared by every checkout on the machine; the script starts one that is missing or stopped, and never stops or recreates one that runs;
2. launches the backend, the generic English application in `be/niord-app/`, with the JVM in UTC and in the `development` mode, which both applications show as a ribbon;
3. starts the development server of the administration application, the public site, or both, so that the frontend builds while the backend starts;
4. once the backend is up, applies the base product's bootstrap to a new, empty database, once (The bootstrap of a new deployment).

With `--sample` the order differs at the end: the script waits for the backend, applies the bootstrap, loads the sample content and restarts the backend, and only then starts the development servers, so that nothing writes to the sample's database between the bootstrap and the load.

The sample content is a set of domains, message series, charts, areas, publications and a few dozen messages from Danish waters, each message titled "SAMPLE —". It lives in `be/niord/sample/` and is loaded only into the local database `niord_sample`; it never reaches a jar, an image or a real deployment.

`Ctrl` + `C` stops the backend and the development servers and leaves the containers running. Their data is kept in named Docker volumes, so the next start continues where you left off. The logs are written to `.local-run/`.

2.3. Ports and addresses

Everything listens on the loopback address only.

What	Port	Address
Administration application (development server)	4100	http://localhost:4100
Public site (development server)	4400	http://localhost:4400
Backend, REST API	8888	http://localhost:8888/rest
Keycloak	8080	http://localhost:8080/auth (admin console at `/auth/admin/`, user `admin`, password `admin`)
MailHog, the caught mails	8025	http://localhost:8025 (SMTP on 1025)
PostgreSQL	5432	database `niord`, or `niord_sample` with `--sample`; user and password `niord`

The development servers pass every request for `/rest` on to the backend (`fe/proxy.conf.mjs`), so the applications and the API share one origin, as they do in a deployment. No CORS configuration exists anywhere, and none is to be added.

2.4. The logins of the local realm

Keycloak imports the realm `be/keycloak/niord-realm-local.json` the first time it starts. Of the realm's groups, four have a local user: one each for the system administrators, the administrators, the editors and the case officers, each with its own name as password and an address on `example.org`. The realm's other groups have no user. The same users sign in to the sample deployment.

User	Password	Group in the realm
`sysadmin`	`sysadmin`	Sysadmin
`admin`	`admin`	Admins

User	Password	Group in the realm
`editor`	`editor`	Editors
`user`	`user`	Case Officers

Keycloak reads the realm file only when the realm does not exist yet. An edited realm file reaches Keycloak only after its container and its volume are removed; `be/README.md` gives the commands.

These passwords are known to everyone. The local stack is for your own machine only: never make it reachable from a network.

2.5. Other ways to run it

- **Only one part.** `be/run.sh` starts the backend and the containers alone; while it runs, `be/reload.sh` compiles your changes and restarts the application in a few seconds. It does not bootstrap the database: a new database stays empty, with no domain, until it has been bootstrapped once, either by `./start-local.sh` or by hand with `be/scripts/bootstrap.sh`, which needs a system administrator's access token (`be/README.md` gives the command). In `fe/`, `npm ci` and then `npm start` serve the administration application on port 4200, and `npm run start:info` the public site on 4400, both against the backend on 8888.
- **Several branches side by side.** `./new-worktree.sh <branch>` makes a git worktree with a slot of its own, so that its ports do not collide with the main checkout's; `./start-local.sh --fe-only` in it runs only the frontend against the backend that already runs. The repository's `README.md` explains the slots.
- **Kubernetes.** `be/k8s/README.md` runs the same stack on a local Kubernetes cluster with the Helm chart (The Helm chart).

A country runs from its own repository with its own `./start-local.sh`, which the country kit writes for it; it uses the same shared containers.

Chapter 3. Backend conventions

The backend is consistent about its names and its idioms, and knowing them makes the code much easier to read. This chapter carries the conventions of the earlier guides forward where the code still follows them, with today's module, package and framework names. The working rules in full are in `be/AGENTS.md`, section *Idioms that matter*.

3.1. Modules and packages

All backend code is in one library, `be/niord/` (`org.niord.base:niord-base`). What earlier versions split into the three projects niord-model, niord-core and niord-web are now three packages of that library:

Package	What it holds
`org.niord.model`	The Niord message model: the value objects the public REST API returns, such as `MessageVo`, `AreaVo` and the GeoJSON classes, and the helpers they use (`DataFilter`, `ILocalizable`). It is not published as a separate artifact.
`org.niord.core`	The JPA entities, the services with the business logic, the batch jobs, the settings, the templates and scripts engine, and the system model value objects that extend the public ones. One sub-package per entity, such as `org.niord.core.area`.
`org.niord.web`	The REST controllers used by the two web applications and by the public REST API (`org.niord.web.api`).

The generic application, `be/niord-app/`, holds only its application class (`org.niord.app.NiordApplication`) and the dependencies a deployable needs and a library does not ship, such as the JDBC driver, Liquibase and the message broker. A country's application is built the same way on the same library.

The test harness the integration tests share (`org.niord.test`, Testing the backend) is published with the library as its test-jar, so that a country's tests can use it too.

3.2. Classes and naming conventions

Each *logical entity* of Niord, such as an area, a chart, a message or a publication, is implemented by a JPA entity, value objects that can be serialised to JSON, a service with the business logic, a REST controller and, for most entities, a batch job that imports them. Take the *Area* entity, one area in the tree of areas:

Class	Description
`AreaVo` (`org.niord.model.message`)	The message model's representation of an area. The `Vo` suffix marks every *value object*: a version of an entity that can be serialised to JSON.
`AreaDescVo` (`org.niord.model.message`)	A language code and all localisable attributes of `AreaVo`. Such classes carry the `Desc` suffix.

Class	Description
`Area` (`org.niord.core.area`)	The JPA entity. Its attributes correspond closely to those of `AreaVo`, but the entity may hold more, which are not part of the public model; the entity classes thus make up a *system model*.
`AreaDesc` (`org.niord.core.area`)	The JPA entity for the localisable attributes of an area.
`SystemAreaVo` (`org.niord.core.area.vo`)	When an entity holds attributes the public message model leaves out, a value object with the `System` prefix carries them. `SystemAreaVo` extends `AreaVo` with them, and it is what the administration application reads and writes.
`AreaService` (`org.niord.core.area`)	The business logic and the life cycle of the entity: a Spring `@Service` with the `Service` suffix.
`BatchAreaImportReader`, `BatchAreaImportProcessor`, `BatchAreaImportWriter` (`org.niord.core.area.batch`)	The three steps of the batch job that imports areas. The classes carry the `Batch` prefix and the `Reader`, `Processor` and `Writer` suffixes. The job itself is declared in `META-INF/batch-jobs/area-import.xml` and run by JBeret; each step is a Spring bean annotated `@Component`, `@Scope("prototype")` and `@Named`, by which name the job's XML refers to it.
`AreaRestService` (`org.niord.web`)	The REST controller, a `@RestController` with the `RestService` suffix, mapped under `/rest/areas`. It is usually a thin layer over the service and decides which value object to return. Every one of its handler methods is covered by a security annotation, on the method or on the class: the class is annotated `@PermitAll`, which its `GET` handlers rely on, and every other handler carries a `@RolesAllowed` of its own (Security).

A few classes keep a name from before these conventions: `RepositoryService`, for instance, is a REST controller (`/rest/repo`). Follow the code around you when you add a class.

3.3. Localisation

Almost every entity can be localised to any number of languages. An entity is associated with a list of objects, each holding a language code and all the localisable attributes in that language. An area has one localisable attribute, its name:

```
public class AreaVo implements ILocalizable<AreaDescVo>, IJsonSerializable {
    List<AreaDescVo> descs;
}

public class AreaDescVo implements ILocalizedDesc, IJsonSerializable {
    String lang;
    String name;
}
```

3.4. Converting between entities and value objects

Every entity converts to and from its value object. In the simplest form, the entity has a constructor that takes the value object, and a `toVo()` method that returns it:

```
@Entity
public class Domain extends BaseEntity<Integer> {
    public Domain() {}
    public Domain(DomainVo domain) {
        // the entity from the value object
    }
    public DomainVo toVo() {
        // the value object from the entity
    }
}
```

`Area` is more involved. It has two value objects, `AreaVo` and `SystemAreaVo`: a message returned by the public API carries the first, and the administration application's area editor reads and writes the second. So `Area.toVo()` takes the class of the value object it returns as a parameter, and the REST controller decides which one to ask for.

The controller also decides which fields to fill in. Most calls return the localised attributes of one language only, which saves bandwidth and keeps the clients simple. And the tree of areas is returned in different shapes: an area attached to a message carries its *parent* ("Kattegat" with its parent "Denmark"), while the areas page of the administration application asks for the root areas with their *children*. The helper class `DataFilter` (`org.niord.model`) says which fields and which language to include. A field may be prefixed with the entity it applies to, so one filter controls the conversion of a whole tree of related entities:

```
DataFilter filter = DataFilter.get()
    .fields("Message.details", "Message.geometry", "Area.parent", "Category.parent")
    .lang("en");
```

The conversion code of `Area` then reads:

```
@Entity
public class Area extends TreeBaseEntity<Area> implements ILocalizable<AreaDesc> {
    public Area() {}
    public Area(AreaVo area, DataFilter filter) {
        // the entity from the value object
    }
    public <A extends AreaVo> A toVo(Class<A> clz, DataFilter filter) {
        // the value object of the given class from the entity
    }
}
```

3.5. Services, controllers and injection

- A service is a Spring `@Service`. Most extend `BaseService` (or `TreeBaseService` for a tree, such as areas), which gives them the entity manager and common queries.
- A controller is a `@RestController` whose dependencies are `private final` fields set by a package-private constructor, as in `AreaRestService`. Newer services are written the same way; older ones inject their fields with `jakarta.inject.Inject`. Spring's `@Autowired` is not used in the main code.
- Loggers are static fields.
- Work at start-up that touches the database runs in a method annotated `@EventListener(ApplicationReadyEvent.class)` and `@Transactional`, never in a `@PostConstruct` method; pin its `@Order` when other start-up work reads the rows it writes.

3.6. Transactions

Transactions are Spring's: `org.springframework.transaction.annotation.Transactional`. `BaseService` is transactional at class level, and many controllers are too. Because Spring applies a transaction through a proxy, a method that calls another method of its own class does not get the other method's transaction settings. When a piece of work needs a transaction of its own, it goes through a separate bean that runs it, as `PublicationTxRunner` does for publications.

3.7. Security

Sign-in is Keycloak's. The backend accepts a bearer token Keycloak issued, and a request names the domain the user works in with the `NiordDomain` header; the user's roles in that domain are added to the request by `NiordDomainFilter`. The roles are `user`, `editor`, `admin` and `sysadmin` (`org.niord.core.user.Roles`), plus `publication-curate`, the right to add or remove the members of a publication issue by hand.

The HTTP layer lets every request through (`SecurityConfig`); method security is the whole guard. So every handler method of every controller resolves to exactly one of the JSR-250 annotations, on the method or its class:

- `@RolesAllowed({ Roles.EDITOR })` and the like, with the role names without a `ROLE_` prefix;
- `@PermitAll` for an endpoint that is public on purpose;
- `@DenyAll` for an endpoint that nobody may call.

An annotation on the method takes precedence over one on its class, and a method or class that carries two of them fails.

A handler without one would be public by omission. The unit test `RestControllerSecurityAnnotationTest` fails the build when one exists, and at start-up `CountryContractGuard` refuses to start an application with such a handler, as it refuses any CORS configuration. The frontends reach the backend on their own origin, so none is needed.

3.8. Errors

A controller signals an HTTP error by throwing `NiordWebException` (`org.niord.core.web`), with a status and a message for the reader:

```
throw new NiordWebException("No area with the id " + id, 404);
```

The exception is a Spring `ResponseStatusException`, so the response is an RFC 9457 problem detail (`application/problem+json`) with the message as its `detail`.

3.9. Settings and Spring properties

Niord has two kinds of configuration, and they never mix:

- **Spring properties** (`application.yml` and its overrides) configure the application as it starts: the database connection, the token issuer, the mail host. They are read at start-up and never written to the database.
- **Settings** are Niord's own. They are stored in the database, and a system administrator edits them on the Settings page. A system property or an environment variable with the same name as a setting takes precedence over the stored value, which is how a deployment fixes one.

A setting is declared in code, never as a bare key string: either as a field annotated `@Setting` (`org.niord.core.settings.annotation.Setting`), which is then injected with its value, or as a static `Setting` constant. The declaration gives the type, the default, the description and whether the setting is shown to the web applications (`web`), editable on the Settings page (`editable`), an operator's value or one the system writes itself (`scope`), and whether every deployment must set it (`required`):

```
@Setting(value = "repoCacheTimeout", defaultValue = "5", type = Type.Integer,
         description = "Cache timeout of repo files in minutes")
Integer cacheTimeout;
```

The declared settings are listed in `be/niord/src/main/resources/defaults/base/settings-catalog.json`, which is regenerated after a declaration changes; `SettingsCatalogCurrentTest` fails when it is stale. A setting only the frontends read goes into `FrontendSettings`. Start-up adds a setting a database lacks and never changes one that exists (Code, data and configuration). A settings file in the repository never holds a real secret: a password setting is empty there.

3.10. Database, JSON and time

- **Schema.** Liquibase builds the schema at start-up (`be/niord/src/main/resources/db/changelog/db.changelog-master.yaml`), and Hibernate then validates the entities against it. A schema change is a new changeset; a changeset that has been applied is never edited, not even in a comment (`ChangelogChecksumPinTest` pins their checksums).
- **Native SQL.** Identifiers are quoted and in mixed case, as in `public."PublicationSeries"`; write

native SQL that way.

- **JSON.** Jackson 3 (`tools.jackson.*`). Jackson 2 is on the class path only for a few libraries that need it; do not use it in new code.
- **Files.** The files of the repository, such as attachments and publication files, are rows of the database, read and written through `RepoStore`. The folder `niord.home` holds only what can be made again: thumbnails, map images, map tiles, the message index and the files of batch jobs. Nothing writes a durable file to disk.
- **Time.** The JVM runs in UTC, and the time zone of the domain formats a time for display. No code may depend on the JVM's default time zone.

Chapter 4. Testing the backend

The backend has two test suites, told apart by the class name. How they run, which containers they start and how the test database is built is described in `be/docs/test-infrastructure.md`; how tests are written here is in `be/docs/test-authoring-guide.md`. This chapter is the overview.

4.1. Unit tests and integration tests

Kind	Class name	Maven plugin	Needs
Unit test	`*Test`	Surefire	nothing but the JDK
Integration test	`*ITTest`	Failsafe	Docker, for the containers the tests start

A class whose name ends in neither is never run. An integration test extends the abstract class `org.niord.test.ITTest`, which boots the Spring application against containers that Testcontainers starts on dynamic ports, so a test never depends on the local stack of Running Niord locally:

- PostgreSQL with PostGIS, for the database,
- Keycloak, with the test realm, which issues the tokens the tests sign in with,
- MailHog, which receives the mails the application sends.

The message broker is not a container: the application under test uses its embedded ActiveMQ Artemis broker, as a deployment does, and a test that reads the messages it sends connects to that embedded broker (`org.niord.test.JmsTestClient`).

The test database is built the way a deployment's is: Liquibase builds the schema, and then a fixture is loaded on top of it, a dump of a database that holds the base product's bootstrap and the sample content (`be/niord/src/test/resources/db/niord-base-data.sql`). The fixture is never edited by hand; `be/niord/scripts/regen-test-seed-postgres.sh` makes it again after the schema, the bootstrap or the sample changed.

4.2. Running the tests

From `be/`:

```
./mvnw clean test                                        # the unit tests, no Docker
./test-it                                                # unit and integration tests
./test-it --its-only -pl niord -Dit.test=KeycloakITTest   # one integration test class
./mvnw -pl niord test -Dtest=AreaToVoTest                # one unit test class
```

Run the integration tests with `./test-it`, not with a bare `./mvnw verify`. When the tests cannot reach Docker, every integration test skips itself, and Maven still reports BUILD SUCCESS with "Tests run: 0". `./test-it` reads the test reports afterwards and fails such a run; it is the same check the continuous integration runs. When a suite skips, `./dev doctor` tells you which Docker endpoint is missing.

A green unit run prints only Maven's own lines: what the tests write to the console goes to `<module>/target/surefire-reports/<class>-output.txt`. A failure is printed in full.

4.3. Writing tests

- In a test, Spring's `@Autowired` and `@PersistenceContext` are the way to inject.
- A test annotated `@Transactional` is rolled back at its end. Data a test needs committed goes through `org.niord.test.Tx.requiringNew()`.
- A test that must also run in every country's build carries `@CountryBuildGroup`. The security tests and the checks of the country contract are in that group, and a country runs them from the library's test-jar.

4.4. The tests that guard the rules

Several unit tests do not test a feature but hold the code to the rules of the earlier chapters. When one of them fails, its message says what to change. Among them:

Test	Fails when
`RestControllerSecurityAnnotationTest`	a REST handler has no security annotation, or names an unknown role
`ChangelogChecksumPinTest`	a Liquibase changeset that has been applied was edited
`RequiredTemplatesTest`	a template the code names is missing from the base product's bootstrap
`SettingsCatalogCurrentTest`	the catalogue of settings no longer matches their declarations
`ReleaseFilesCurrentTest`	the generated files of the bootstrap or of a release folder are stale (Code, data and configuration)
`CountryWordsTest`	the base product's main code, its bootstrap or the generic application names a country word, a Danish locale in a template, or a Danish or Faroese place name, outside the exceptions its list allows with a reason

Chapter 5. Frontend architecture and conventions

The frontend is one Angular workspace in `fe/`, with one `package.json`, one lock file and one set of tool configurations, holding two applications that ship separately. This chapter is a map of the workspace and says where its conventions are written down; `fe/README.md` is the entry point to all of them.

5.1. A map of the workspace

Path	What
`fe/ui-apps/admin/`	The administration application (Angular project `niord-app`): messages and their editor, the firing-area schedule, and the administration and system administration pages, behind a Keycloak sign-in. Its end-to-end tests are in `fe/ui-apps/admin/e2e/`.
`fe/ui-apps/info/`	The public site (Angular project `niord-info`): the published messages as details, list, map and print, the publications, and a small map ("teaser") other sites can embed. No sign-in, read-only, and held to a size budget.
`fe/styles/`	The design tokens, the bridge to Tailwind's theme, the named classes both applications render and the rules for the HTML a message carries.
`fe/shared-map/`	The OpenLayers map kit both applications use, imported as `@niord/map`. Its `portrayal/` folder defines how messages are drawn on a map; the backend reads the same definition when it draws a message's map image itself, so a change there is a backend change too.
`fe/shared-html/`	The sanitiser for the HTML a message carries, imported as `@niord/html`.
`fe/languages/`	The base product's one language pack, English. Every other language is a pack in a country's repository (Building a country).
`fe/scripts/`	The workspace's checks and tools, run by the npm scripts.
`fe/docs/`	The documents listed below.

Neither application imports the other; what they share lives in the three shared folders.

5.2. The layers of the administration application

The code of the administration application, under `fe/ui-apps/admin/src/app/`, has four layers, and ESLint rules enforce which may import which:

- `core/`: what the whole application depends on, one instance each: sign-in, configuration, the current domain, translations, HTTP interceptors, the main shell.

- `shared/`: building blocks without knowledge of any domain area: components, dialogs, forms, pipes, the wire models.
- `features/`: the domain areas, each loaded when first used: `home/`, `messages/`, `editor/`, `schedule/` and `admin/`. `messages/` is the one area the others may build on.
- `country-specific/default/`: the neutral defaults a country's files replace: the language list, the country's providers, routes and menu entries.

Data flows one way: an API service owns an HTTP call and maps the wire format to the model, a facade turns it into signals for one screen, and a component renders the signals. A component that calls the API itself is in the wrong layer. A country's code imports the base product only through the application's public API (`@niord/admin/api`, `@niord/info/api`), and the base product never imports it.

5.3. Where the conventions are written down

Document	What it covers
`fe/docs/architecture.md`	the layers and their import boundaries, the data flow, state, extension points, the map layer
`fe/docs/angular-conventions.md`	signals and effects, how data is loaded (`httpResource`, `HttpClient`, `rxResource`), Signal Forms, templates, routing, translations
`fe/docs/typescript-conventions.md`	compiler flags, what lint enforces, naming, wire types against model types
`fe/docs/coding-standards.md`	error handling, dates and time zones, comments, dependencies
`fe/docs/ui-standards.md`	the shared components, overlays, form controls, accessibility, responsive layout; why no component library is used
`fe/docs/styling-reference.md`	every design token and named class, and where each lives
`fe/docs/platform.md`	start-up order, configuration and the runtime settings, sign-in, domains, the service worker
`fe/docs/testing.md`	the unit-test harness and its traps, the end-to-end fixtures
`fe/docs/definition-of-done.md`	what a change has to pass before it is done, and what enforces each check
`fe/docs/country-deployments.md`	the seams a country fills and their neutral defaults
`fe/AGENTS.md`, `fe/ui-apps/admin/AGENTS.md`, `fe/ui-apps/info/AGENTS.md`	the working rules of the workspace and of each application; the public site's rules, its checks and its container are in its own file

5.4. Everyday commands

From `fe/`, after `npm ci`:

```
npm start              # the administration application on http://localhost:4200
npm run start:info     # the public site on http://localhost:4400
npm test               # the administration application's unit tests (Vitest)
npm run test:info      # the public site's unit tests
npm run lint           # ESLint over the administration application
npm run verify         # every check of the administration application, as CI runs
them
npm run verify:info    # every check of the public site, as CI runs them
```

Every `ng` command names its project (`ng build niord-app`, `ng serve niord-info`); in a workspace with two projects a bare `ng build` stops. Prefer the npm scripts above: each start and build script first stages the language pack into `.niord/build/<app>/`, which is all a build reads of the translations. An `ng serve` or `ng build` run directly needs `npm run languages:stage` first; without it the application gets no translations, or those of an earlier staging. A pre-commit hook formats and lints the files you stage. The end-to-end tests need a running backend and Keycloak, and are not part of the continuous integration; `fe/README.md`, section *End-to-end tests*, says how to run them.

Chapter 6. Code, data and configuration

A running deployment's data belongs to its operators. Niord 4 therefore keeps three things apart, and the rules of this chapter decide what a change to the repository may do to a deployment. The rules in full are in `be/AGENTS.md` (*Code, data and configuration*); the reasoning behind them is in `docs/code-data-config-plan.md`.

6.1. Three kinds of thing

- **Data** is everything an operator can see and edit in the administration application: domains, message series, areas, categories, and also every template, script and report, the built-in ones included. Data lives in the database.
- **Code** is everything else that the repository ships: the Java and TypeScript code, the default texts and the default settings files.
- **Configuration** is the Spring properties a deployment sets. They are read at start-up and never written to the database.

The templates are data, not code: no jar or image holds one, and the backend reads every template and script from the database only.

6.2. What a change to the repository may do to a deployment

- A change never changes or deletes a deployment's rows. It may only add keys a deployment never had: new text keys and new setting keys. Start-up adds such a key where it is missing, and never touches it again; an operator's value stays.
- Nothing is reloaded and nothing is reset. A read never writes a row, there is no reload of a template from a jar and no reset to a default, and what an operator deleted stays deleted. The backend remembers which default texts it added and which an operator deleted (`DefaultsLedger`), so a deleted text does not come back at the next start.
- A Liquibase changeset changes the schema, and changes values only as far as a schema change requires. It never inserts content, deletes rows or changes what a value says.
- A template path the code relies on is listed in `RequiredTemplates`; a template a deployment does not have is reported by the missing-path report at start-up and at `GET /rest/script-resources/missing-paths`, never created silently.

Data reaches a deployment in two ways only: by the bootstrap of a new database, and by a system administrator's import.

6.3. The bootstrap of a new deployment

A new database holds no data at all, not even a domain. The backend builds the schema and adds the default settings and English texts at its first start, and nothing else. The database is then *bootstrapped* once, with the base product's bootstrap:

- The bootstrap is the folder `be/niord/bootstrap/`, published with every version as `org.niord:niord-bootstrap:<version>:zip`. It holds the active master domain, all the templates and scripts, the base categories (among them `Firing Exercises`, whose English name the firing schedule depends on), the parameter types that are not standard, the built-in reports, the promulgation types and a data dictionary. The standard parameter types are part of the code and held in memory, so the bootstrap may not contain them. Its data is in English only. Its `README.md` lists the files.
- A country's own bootstrap follows the base product's in the same run and adds the country's domains, series, languages and templates. Within that run a later archive's values win.
- The bootstrap command is `POST /rest/bootstrap`, which `be/scripts/bootstrap.sh` wraps; it needs a system administrator's token. `./start-local.sh` runs it by itself on a new local database.
- A bootstrap runs once, on an empty database. The command refuses a database that holds data, naming the first table that holds a row, and a database bootstrapped before. `GET /rest/bootstrap` tells the state: `pending`, `existing`, `in-progress`, `applied` or `failed`. A failed bootstrap is not retried: the remedy is a database recreated empty and the same command with the same archives.

Data a new deployment needs, such as a template the code names or a built-in report, goes into the bootstrap, never into code that writes it at start-up.

6.4. Release folders and a system administrator's import

A deployment that already runs gets a new or changed template only when its system administrator imports it. A version that changes a template ships the change in a *release folder*, `be/niord/bootstrap/releases/<yyyy-mm-dd>-<topic>/`, which holds:

- `script-resources.json`, the templates and scripts at that release, for the import on the Script resources page;
- `texts.json`, the base product's English texts at that release, for the import on the Dictionaries page;
- `missing-paths.sql`, the missing-path report as a read-only query;
- a `README.md` that says what changed and which overwrites to tick.

The generated files are written by `ReleaseFilesGenerator` (`be/niord/bootstrap/README.md` gives the command), and a release folder never changes once it is merged into the main branch. A change in Java that a template depends on stays compatible with the templates deployments already hold.

The imports on the Script resources and Dictionaries pages show a preview first. They add what is missing and overwrite only what the system administrator ticks, and they record who imported what. Whether to import at all is the system administrator's decision; nothing applies a release folder by itself. How a system administrator does it is described in the User Manual for Sysadmins.

6.5. Defaults, the sample and the repository files

- The base product's default texts and settings are under `be/niord/src/main/resources/defaults/base/`, a country's under `defaults/country/`. Texts are layered key by key, and a key is added only where it is missing. A text key Java reads is listed in `RequiredTexts`.
- The sample content of `be/niord/sample/` is loaded only into the local database `niord_sample` (Running Niord locally). It is never in a jar or an image; a check in the publication run makes sure of that.
- The files of the repository (attachments, publication files) are rows of the database. The folder `niord.home` on a server is a cache that nothing backs up: everything on it can be made again. The database is the one thing to back up.

Chapter 7. Building a country

A deployment of Niord is always a country: a repository of its own, built on one pinned version of the base product (The base product and its countries). The base product itself has no country's code, texts or data; its generic English application in `be/niord-app/` is a reference, not a deployment.

How to build and run a country is written up in the *country guides*, which ship with the country kit and match the version a country builds on. The full set is read in the kit a country builds with, or in `country-kit/` of the base product's repository (Where the rules are). Only part of it is published on this site, among it the overview of the guides, the operators' guide and the designers' brief: The country kit's guides. The country guide itself and the kit's reference are not on the site. This chapter only says where the guides are and how the pieces fit; it does not repeat them.

7.1. What a country repository holds

- **The pins.** The base product version is the parent version in the country's `be/pom.xml`. The file `fe/niord-pins.lock.json` pins the same version's frontend source zip and country kit by their checksums, and any language packs the country takes from other countries.
- **The backend.** The country's application is the base library, `org.niord.base:niord-base`, plus its own settings file, logos, texts and, where needed, beans at the extension points the backend contract allows.
- **The frontend.** The country's set-up, `node setup.mjs`, fetches the base product's frontend source zip and the kit, checks them, and lays the country's files over the base product at the *seams* it offers: the manifest, branding and colours, languages, text overrides, and at most a few pages, hooks and slots of its own. Everything else is generated from the base product on every set-up. Where no seam serves, a country may replace one base product file of an application's `src/app/` with its own copy, a *guarded override*: it is made with `node setup.mjs --add-override <file> --reason <text>` and listed in the manifest's `overrides`, and the build fails whenever the base product changes the original, until the country has compared its copy again. An override that is still needed after a version or two is a request for a new seam.
- **Its data.** The country's own bootstrap (`be/bootstrap/`) and release folders, applied after the base product's (Code, data and configuration).

A new version of the base product reaches a country as one pull request in the country's repository, which moves the pins; the country runs its set-up and its tests on it, and a person reviews and merges it.

7.2. Where the rules are

In the base product's repository:

- `country-kit/` is the kit itself. Its `README.md` is the reference for set-up, the seams, the language packs and the checks on every country build.
- `country-kit/docs/` holds the country guides: who does what and how an upgrade works, the country guide with its in-depth companion, the designers' brief, the operators' guide, and a prompt for a coding assistant that sets up a country.

- `country-kit/API-CHANGES.md` records what each base product version changed for a country and what a country must do about it.
- `be/docs/country-api.md` is the backend contract: the extension points, the beans and properties a country may not replace, and how both are enforced. It ships in the kit as well.
- `fe/docs/country-deployments.md` describes the frontend's seams and their neutral defaults.
- `country-kit/example/` is an example country, `xx`, a complete country repository in miniature. `node ../country-kit/example/run.mjs --work ../.local-run/example`, run from `fe/`, sets it up and builds it against the working tree; the continuous integration does the same on every change the example depends on.

When a change to the base product changes what a country may use or must do, it is recorded in `country-kit/API-CHANGES.md` in the same change.

Chapter 8. Releases, images and deployment

Nobody assembles a release of Niord by hand. Every merge to the main branch that changes code and whose checks all pass is published as a new version, and the countries take the versions they want from there. The workflow that does it is `.github/workflows/publish.yml`; `be/docs/cicd/intended-use.md` explains the whole path from a change to a running deployment in plain words.

The publication starts when the checks of a push to `main` finish. A merge that changes only documentation, such as the guides of this site under `guides/`, the backend's documents under `be/docs/` or the records under `docs/`, starts no checks on `main`, so it publishes nothing; its change is carried by the next version a code merge publishes.

8.1. What a release consists of

A release is a version number, `4.0.<n>`, where `<n>` is the number of the publication run. One version covers the whole base product, backend and frontend together, so a backend and a frontend that were never tested together cannot be combined. A version is published only when every check that ran for its commit on `main` passed; a red check publishes nothing. There is one exception, for an emergency: a maintainer can start the publication by hand on `main`, and that run publishes after the backend's unit tests alone, while the integration tests run and report afterwards. Such a run is labelled as one in its name and summary, and it is not for routine use.

Each version publishes:

Artifact	What it is
`org.niord:niord-parent`	The parent POM a country's backend inherits: the dependency versions the library was tested with and the test configuration.
`org.niord.base:niord-base`, with its test-jar	The backend library, and the test harness a country's tests use.
`org.niord:niord-bootstrap` (zip)	The base product's bootstrap (The bootstrap of a new deployment).
`org.niord.web:niord-web-admin` (zip, classifier `en`)	The production build of the generic administration application.
`org.niord.web:niord-fe-src` (zip)	The base product's frontend sources, which a country's set-up builds from.
`org.niord:niord-country-kit` (zip)	The country kit, with its guides and the backend contract.
`org.niord:niord-image-tools` (zip)	The scripts that build a backend image and check a country's jar.
`org.niord:niord-local-run` (zip)	What a country's `start-local.sh` needs for a local run: the launcher library, the compose file, the local realm and the sample content. It holds known passwords and is for a local run only.

Artifact	What it is
The image `niord/niord-app-en`	The container image of the generic application (The images).
The Helm chart `niord`	The chart that deploys a stack (The Helm chart), with the same version as the application.

The Maven artifacts go to the GitHub Packages registry of the NiordOrg organisation, the image and the chart to the project's container registry. The commit is then tagged `v4.0.<n>`, and a release on that tag records the SHA-256 checksums of the source zip and the kit package, which a country pins. Before anything is published, a check makes sure that the backend's jars, the bootstrap zip and the inputs of the image carry nothing of the sample content; `niord-local-run` is the one artifact that carries the sample, on purpose, for a local run.

A published version can be withdrawn with the workflow `.github/workflows/retract-version.yml`. It removes the version's Maven artifacts, its release and its tag; the image and the chart in the container registry it only lists, and they are deleted there by hand. A country that pinned the version is then told to move on.

8.2. The images

The backend image is built from `be/k8s/docker/Dockerfile`, one image per deployable: the generic one here, and each country's in its own repository from the same scripts. The image runs the application on Java 25 as an unprivileged user and listens on port 8888. Its layers are ordered so that the parts that rarely change are stored once: the Java runtime, then the third-party libraries (built to be the same bytes on every build), then `niord-base`, the application and, last, the production build of the administration application. The backend serves that build at the root of its own origin, so a deployment needs no separate web server for the administration application. To build the image locally, run `be/k8s/scripts/build-image.sh` (it needs `npm ci` run once in `fe/`).

The public site's image is built from `fe/ui-apps/info/Dockerfile`: nginx serving the built files as user 101 on port 8080, passing only the public, read-only `/rest` paths on to the backend. Each country builds and publishes its own, from its own build of the site, with `npm run image:info`; the base product publishes none. In the base product the command is a manual tool for trying the image locally.

8.3. The Helm chart

The chart in `be/k8s/helm/niord/` deploys one complete stack, one release per country in that country's own namespace. It has six components, each behind an `enabled` flag:

Component	What
`proxy`	nginx, the single public entry point: it sends `/auth` to Keycloak and everything else to the backend.
`app`	The backend, which also serves the administration application.
`postgres`	PostgreSQL with PostGIS.

Component	What
`keycloak`	Keycloak. A cluster can instead use a Keycloak outside the release, shared by several deployments.
`mailhog`	MailHog, which catches mail on test environments.
`info`	The public site, off by default; an environment turns it on with its country's image, under the same version as the backend, on a host of its own.

```
browser --> https://<deployment host>  --> proxy (nginx)
                                             |-- /auth/*  --> keycloak
                                             `-- the rest --> app: /rest, the
                                                              administration
application

reader  --> https://<public site host> --> info (nginx) -- public /rest paths only -->
app
```

Every pod runs as a non-root user and meets the Kubernetes Pod Security level `restricted`. Each component's service is named `<release>-<component>`, with `-niord` added to the release name when it does not contain "niord" already: `niord-proxy` in a release named `niord`, `staging-niord-proxy` in one named `staging`. The chart's version is the application's version, and the image tag defaults to it, so pinning the chart pins the images. `be/k8s/README.md` describes the values, the two modes of Keycloak (development, with the test realm, and production), and how to run the whole chart on a local Kubernetes cluster.

8.4. Deployment

The base product deploys nothing itself. Each country takes a new version through a pull request in its own repository (Building a country), builds its images, and deploys them with the published chart to its staging environment. The configuration of each environment is kept in a deployment repository of its own, outside this one. Production is to be changed only by a person, who promotes exactly the version staging has been running; nothing reaches it by itself. A new deployment's database is bootstrapped once (The bootstrap of a new deployment), and its operators then set up its data, as the country kit's operators' guide describes.

Chapter 9. Contributing

This chapter describes how a change travels into the main branch. The rules are kept with the code in the repository's `AGENTS.md` (section *Working here*), `README.md` (section *CI*) and `fe/docs/definition-of-done.md`.

9.1. Branches and pull requests

- The main branch, `main`, is the trunk. Every merge to it that changes code and passes its checks becomes a released version (Releases, images and deployment), so `main` is kept releasable at all times.
- A change is made on a branch of its own and goes into `main` through a pull request. A larger piece of work can collect several smaller pull requests on a long-lived `feature/<name>` branch first, and reach `main` as one pull request at the end.
- Open a pull request as a draft while you are still working on it. A draft runs no checks; marking it ready for review starts them.
- Each new push to a pull request that is ready for review starts the checks again and cancels the run in progress, so push when a piece of work is done rather than after every commit.
- A change to the base product that changes what a country may use or must do is recorded in `country-kit/API-CHANGES.md` in the same pull request (Building a country).
- A bug you find outside the task at hand is written down, not fixed in passing: backend findings in `be/docs/potential-errors.md`, frontend ones in `fe/docs/backlog.md`.

9.2. Continuous integration

The workflows are in `.github/workflows/` at the root of the repository; GitHub reads them nowhere else. The checking workflows run on pull requests into `main` once they are ready for review. On a pull request, each workflow first looks at which files the change touches and skips the work that does not concern it. On a push to `main`, a workflow starts only when the push touches the paths it watches, and `guides.yml` never runs on a push: it runs on pull requests into `main` that touch the guides or their sources, and by hand.

Workflow	What it checks
`be-ci.yml`	The backend: the unit tests and the integration tests, and a country made by the kit's `init` when the change reaches what that country's backend is built from. A change only to the backend's documentation runs nothing.
`fe-ci.yml`	The administration application and the kit: formatting, documentation links, lint, the guard tests, the unit tests and the production build, and the example country built against the change.
`info-ci.yml`	The public site: the same kinds of check, plus its bundle size against a recorded baseline.
`guides.yml`	This site: a change under `guides/` is built and its links are checked.

Workflow	What it checks
`publish.yml`	After the checks of a push to `main` passed: the publication of the version.

9.3. The gates

The branch protection of `main` will require three checks: **Frontend gate** for the administration application, **Info gate** for the public site and **Unit tests** for the backend. The two gates are jobs that summarise their workflow: each is green only when every other job of its workflow succeeded. **Unit tests** is not such a gate. It is the backend's unit-test job itself and says nothing about the integration tests or the generated country, which are jobs of their own beside it. Until the branch protection is switched on, every check is advisory, so read every check, these and the others, before merging.

9.4. Local verification

The checks of the continuous integration cost time, so verify in proportion:

- While you work, run only the tests of what you changed: one backend test class (`./test-it --its-only -pl niord -Dit.test=<class>` in `be/`) or one spec file in `fe/`.
- Run the whole suite once, as the last step before review. For a pull request into `main`, the continuous integration does that.
- A pull request into a `feature/**` branch gets no continuous integration at all. Run `./verify-local --base origin/feature/<name>` at the root of the repository instead, and paste the summary it prints into the pull request. It runs the same steps as the workflows for the parts the branch changed: the whole backend suite with the check that the integration tests really ran, the frontend gates, and the example country when the branch reaches it.

`fe/docs/definition-of-done.md` lists what a frontend change has to pass beyond the gates, such as the end-to-end tests and the manual checks of the public site's image.

9.5. Licence

Niord is open source under the Apache License 2.0, stated in `be/LICENSE` and `fe/LICENSE`. Most of the backend's Java files carry the licence header; the frontend's files carry none, and their licence is stated in `fe/LICENSE` and `fe/README.md` only.